

Práctica 3.1: Introducción a Optimización y explotación Paralelismo en GPUs

Índice

Objetivos	1
Ejercicio 1: Hello World	1
Monitorización de uso de GPU	2
Variables de entorno	4
Ejecución en GPU de NVIDIA	4
Ejemplo 2: mejoras en la programación con OpenMP para explotar el paralelismo heterogéneo	4
Maximizar el uso de recursos	6
Minimización transferencias CPU-GPU	7
Ejercicio 3: Inferencia CNN con OpenMP (entrega)	9
Arquitectura del modelo	9
Tareas del alumno	11
Recomendaciones propuestas	11
Resultado esperado	11

Objetivos

Esta práctica persigue que los estudiantes se familiaricen con la mejora de rendimiento por la explotación de paralelismo heterogéneo en GPUs mediante el uso de OpenMP. Este esquema de paralelismo es compatible otros estudiados hasta el momento en la asignatura, sin embargo nos centraremos en las cuestiones relacionadas con la descarga de trabajos (kernels) en un acelerador del tipo GPU.

Ejercicio 1: Hello World

El primer ejemplo “simple.cpp” inicializa en el host un array `data`, lo duplica en el device (construcción **target**). Entre las cuestiones a tener en cuenta está el movimiento de datos se produce con la clausula **map**. Durante las clases hemos visto como se puede indicar la direccionalidad del movimiento de datos, con(*from*, *to*, *fromto*, ect) por lo que se recomienda al alumnado repasar estos aspectos.

```
...
// Initialization
for (int i = 0; i < N; i++) data[i] = i;

// Add the target directive here, including the map clause.
#pragma omp target map(from : is_cpu) map(tofrom : data [0:N])
{
    is_cpu = omp_is_initial_device();
#pragma omp parallel for
    for (int i = 0; i < N; i++) {
        data[i] *= 2;
    }
}
```

```

    }
}
...

```

En referencia a la compilación, el compilador Intel® oneAPI DPC++/C++ Compiler tiene soporte de OpenMP. La especificación completa de OpenMP está disponible en la página web del estandar de OpenMP. El soporte para GPUs se denomina “offloading” haciendo referencia a la posible descarga de código en el acelerador. El compilador Intel® oneAPI DPC++/C++ tiene soporte de offloading mediante la activación en la fase de compilación con el flag `-fiopenmp -fopenmp-targets=spir64`. (Recordar la activación del compilación cargando las variables de entorno correspondientes con `source /opt/intel/oneapi/setvars.sh`).

Además de forma análoga a las prácticas anteriores se puede activar un fichero de reportes para verificar que el código generado se descarga en el acelerador, añadiendo en fase de compilación el flag `-qopt-report`. En este caso además del fichero “**.optrpt**” también se genera un equivalente con el formato “**-openmp-spir64.optrpt**” donde se indica la descarga a partir de la línea 23.

```

user@lab:~ $ icpx -o simple simple.cpp -fiopenmp -fopenmp-targets=spir64 -qopt-report=2
user@lab:~ $ more simple-openmp-spir64.optrpt
*****
***** Optimization Report - Device Compilation *****
*****
Begin optimization report for: OMP TARGET outline from main line 23
...

```

Monitorización de uso de GPU

Es interesante conocer el proceso de compilación y ejecución del binario, puesto que emplea la técnica de “JIT Compilation” para en tiempo de ejecución generar el binario final para el acelerador o GPU.

Esta fase tiene un impacto directo en la ejecución del código, permitiendo la monitorización de la ejecución con la variable de entorno `LIBOMPTARGET_PLUGIN_PROFILE`. En el ejemplo se muestra este aspecto con la fase de “Compiling”, además de conocer el tiempo que conlleva el movimiento de datos: **DataRead/DataWrite** así como el tiempo del kernel: ****__omp_offloading_10303_16a5834__Z4main_l23****.

```

user@lab:~ $ LIBOMPTARGET_PLUGIN_PROFILE=T ./simple
Num devices=1
Running on GPU
0
2
4
6
8
10
12
14
16
18
20
22
24
26
28
30
=====
LIBOMPTARGET_PLUGIN_PROFILE (LEVEL_ZERO) for OMP DEVICE(0) Intel(R) Graphics, Thread 0
=====
Kernel 0                : __omp_offloading_10303_16a5834__Z4main_l23
=====

```

Just-In-Time (JIT) Compilation Flow

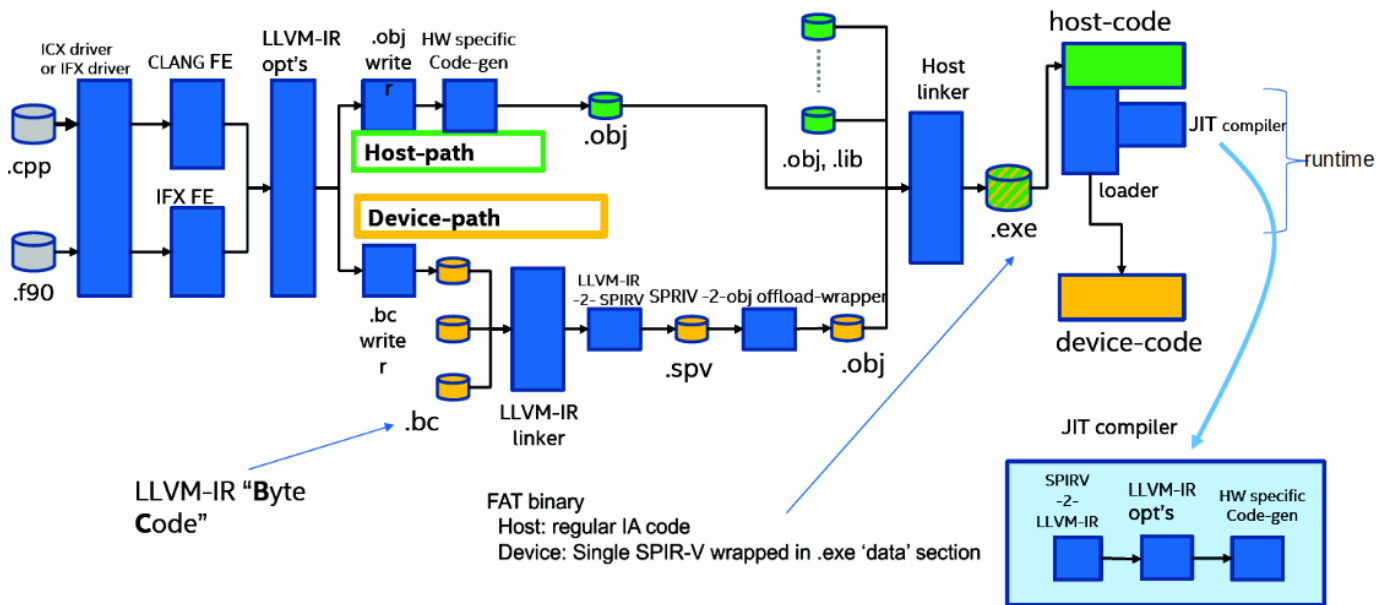


Figure 1: Flujo de compilación y ejecución de código OpenMP-target

Name		Host Time (msec)			Device Time (msec)				Min	
		Total	Average	Min	Max	Total	Average			
Compiling	:	147.31	147.31	147.31	147.31	0.00	0.00	0.00	0	0
DataAlloc	:	2.20	0.22	0.00	1.89	0.00	0.00	0.00	0	0
DataRead (Device to Host)	:	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0	0
DataWrite (Host to Device)	:	0.02	0.02	0.02	0.02	0.00	0.00	0.00	0	0
Kernel 0	:	1.43	1.43	1.43	1.43	0.01	0.01	0.01	0	0
Linking	:	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0	0
OffloadEntriesInit	:	1.94	1.94	1.94	1.94	0.00	0.00	0.00	0	0

Variables de entorno

Existen otras variables de entorno que permiten controlar la ejecución del código en los dispositivos así como activar opciones de depuración:

```
OMP_TARGET_OFFLOAD = mandatory | disabled | default
```

Donde: * mandatory: la región *target* ejecuta en GPU u otro acelerador * disabled: región *target* en CPU * default: región *target* en GPU (si hubiese), sino en CPU

En incluso existen opciones para depuración: * LIBOMPTARGET_DEBUG= [1 | 2] * Más información en el runtime del LLVM

Ejecución en GPU de NVIDIA

También existe soporte de OpenMP en gráficas de NVIDIA, con el SDK para hpc de NVIDIA El compilador NVC++ tiene soporte OpenMP offloading. Para ello, la activación de OpenMP se efectúa con el flag -mp=gpu que a su vez tiene soporte **offloading**. De igual forma la herramienta de profiling NVIDIA Nsight Systems permite monitorizar el tiempo de uso de cada kernel en la GPU de NVIDIA:

```
user@system:~$ /opt/nvidia/hpc_sdk/Linux_x86_64/2024/compilers/bin/nvc++ -o simpleNVC simple.cpp -mp=gpu
user@system:~$ nsys nvprof ./simpleNVC
....
Time (%)   Total Time (ns)   Instances   Avg (ns)   Med (ns)   Min (ns)   Max (ns)   StdDev (ns)   GridXYZ
-----
    100,0           33.024             1  33.024,0  33.024,0   33.024    33.024           0,0       30    1
....
```

Ejemplo 2: mejoras en la programación con OpenMP para explotar el paralelismo heterogéneo

El modelo de ejecución se conoce con el nombre de modelo host-device, donde un *host* (habitualmente una CPU) está conectada a un acelerador o *device* (GPU). Las GPUs tiene diferentes construcciones, y jerarquías de unidades de ejecución. En el caso concreto de las GPU de Intel la distribución se realiza de la siguiente manera: - Cada dispositivo GPU posee varias **GPU compute Units (CE)** - Cada CE posee varias **Unidades de Ejecución (EUs)** o también denominados recientemente **Xe Vector Engines (XVE)** - Existe también un modelo de memoria: memoria Global (memoria de GPU), una **memoria compartida: SLM** a nivel de CE o también denominado Slice - Además cada EUs posee unidades SIMD

En lo referente a OpenMP, existen varias niveles de pragma que permiten expresar el paralelismo: - target: descarga en el acelerador - teams: replica cómputo a nivel de “CE” o “Slice” - distributed: distribuye iteraciones a nivel de “work-group” - parallel for: distribuye iteraciones a nivel de EU - simd: distribuye iteraciones a nivel de sub-group (simdlen=8, 16, 32)

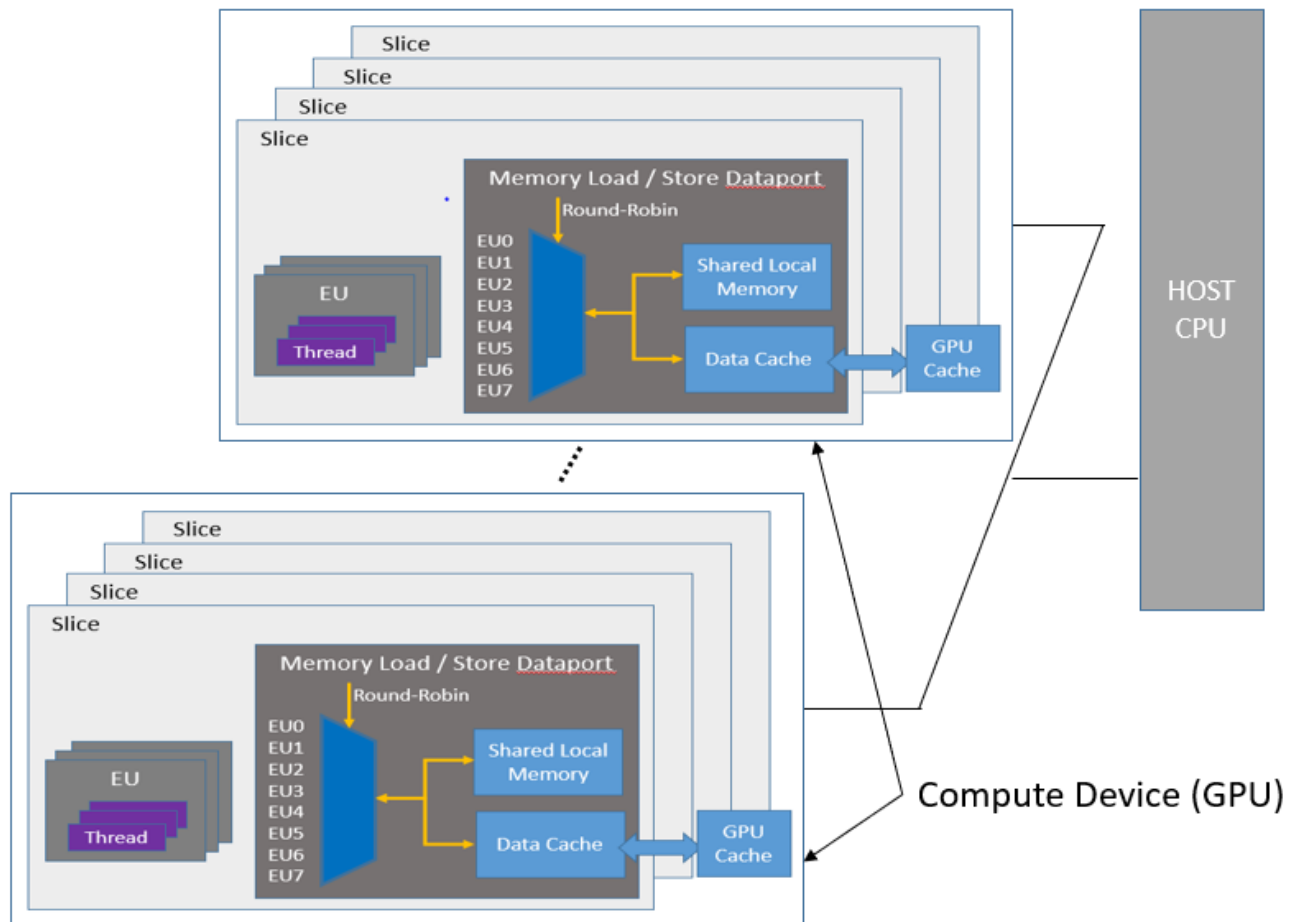


Figure 2: Fig: Modelo de ejecución en GPU

Concept	SYCL	OpenMP	CUDA
Work-item	Work-item	OpenMP thread or SIMD lane	CUDA thread
Work-group	Work-group	Team	Thread block
Work-group size	Work-group size	Team size	Thread block size
Number of work-groups	Number of work-groups	Number of teams	Number of thread blocks
Sub-group	Sub-group	SIMD chunk (simdlen = 8, 16, 32)	Warp (size = 32)
Max work-items per group	Maximum number of work-items per work-group	Thread limit	Maximum number of CUDA threads per thread block

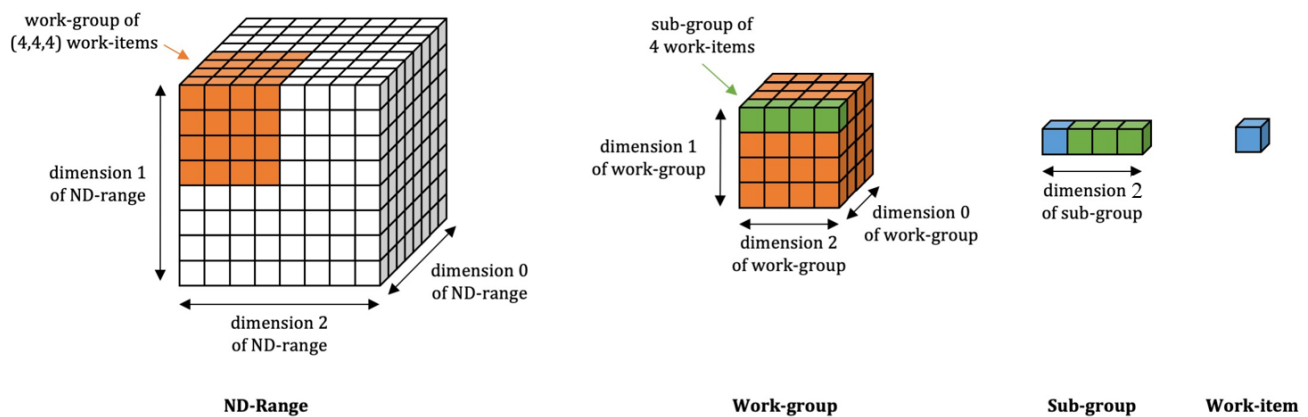


Figure 3: Fig: Relación de ND-Range, work-group, sub-group, work-item

Maximizar el uso de recursos

El código “test_no_collapse.cpp” aplica la paralelización a nivel de bucle `for` (`b = 0; b < BLOCKS; b++`) siendo **BLOCKS 4**

```

...
/* offload the kernel with no collapse clause */
#pragma omp target teams distribute parallel for \
    private(b, i, j, k, l)
for (b = 0; b < BLOCKS; b++) {
    for (i = 0; i < P; i++) {
        for (j = 0; j < P; j++) {
            for (k = 0; k < P; k++) {
                float ur = 0.;
                float us = 0.;
                float ut = 0.;

                for (int t=0 ; t < TIMES; t++)
                    for (l = 0; l < P; l++) {
                        ur += dx[IDX2(i, l)] * u[IDX4(b, l, j, k)];
                        us += dx[IDX2(k, l)] * u[IDX4(b, i, l, k)];
                        ut += dx[IDX2(j, l)] * u[IDX4(b, i, j, l)];
                    }
            }
        }
    }
}

```

```

    }
    w[IDX4(b, i, j, k)] = ur * us * ut;
  }
}
}
...

```

Evaluando la ejecución con variable entorno `LIBOMPTARGET_DEBUG=1` las iteraciones del bucle=4 porque `BLOCKS=4` se observa que el número de teams es **Number of teams = {4, 1, 1}**, es decir que **solamente 4 EUs están siendo utilizadas simultáneamente**

```

user@system:~$ icpx -fiopenmp -fopenmp-targets=spir64 test_no_collapse.cpp
user@system:~$ OMP_TARGET_OFFLOAD=MANDATORY LIBOMPTARGET_DEBUG=1 ./a.out
...
Target LEVEL_ZERO RTL --> Assumed kernel SIMD width is 16
Target LEVEL_ZERO RTL --> Preferred team size is multiple of 32
Target LEVEL_ZERO RTL --> Loop 0: lower bound = 0, upper bound = 3, Stride = 1
Target LEVEL_ZERO RTL --> Team sizes = {1, 1, 1}
Target LEVEL_ZERO RTL --> Number of teams = {4, 1, 1}
...

```

Si añadimos las cláusulas *collapse* se mejora utilización de recursos. Por ejemplo en el código “test_collapse2.cpp” se añade la cláusula **collapse(2)**, las iteraciones del bucle=8 porque `BLOCKS*P = 4*8 = 32 * SIMD: 16 * Team sizes: {1,1,1}` y **Número de teams: {8,4,1}** luego **Número total teams: {8,4,1}=32**

```

user@system:~$ icpx -fiopenmp -fopenmp-targets=spir64 test_collapse2.cpp
user@system:~$ OMP_TARGET_OFFLOAD=MANDATORY LIBOMPTARGET_DEBUG=1 ./a.out
Target LEVEL_ZERO RTL --> Assumed kernel SIMD width is 16
Target LEVEL_ZERO RTL --> Preferred team size is multiple of 32
Target LEVEL_ZERO RTL --> Loop 0: lower bound = 0, upper bound = 7, Stride = 1
Target LEVEL_ZERO RTL --> Loop 1: lower bound = 0, upper bound = 3, Stride = 1
Target LEVEL_ZERO RTL --> Team sizes = {1, 1, 1}
Target LEVEL_ZERO RTL --> Number of teams = {8, 4, 1}
...

```

De igual manera están los códigos “test_collapse3.cpp” (*Team sizes: {8,1,1}* y *Número de teams: {1,8,4}*) y “test_collapse4.cpp” (*Team sizes: {32,1,1}* y *Número de teams: {64,1,1}*) mejorando la distribución de paralelismo como se verifica en los tiempos de ejecución:

- time sin-collapse = 0.196160 s.
- time collapse(2) = 0.024447 s.
- time collapse(3) = 0.003280 s.
- time collapse(4) = 0.001072 s.

Minimización transferencias CPU-GPU

Otro aspecto a tener en cuenta a la hora de explotar el paralelismo heterogéneo es la **minimización de transferencias entre memorias del host y device**. El uso del bus PCIe que conecta ambos dispositivos redonda significativamente en el rendimiento final de una aplicación.

El código disponible “test_no_target_enter_exit_data.cpp” muestra el impacto de transferir `u[0:SIZE]`, `dx[0:P * P]`, `w[0:SIZE]` en el kernel#2, cuando ya está disponible en la memoria de la GPU tras la ejecución del kernel#1.

```

...
/* offload kernel #1 */
#pragma omp target teams distribute parallel for collapse(4) \
    map(to: u[0:SIZE], dx[0:P * P]) map(from: w[0:SIZE]) \

```

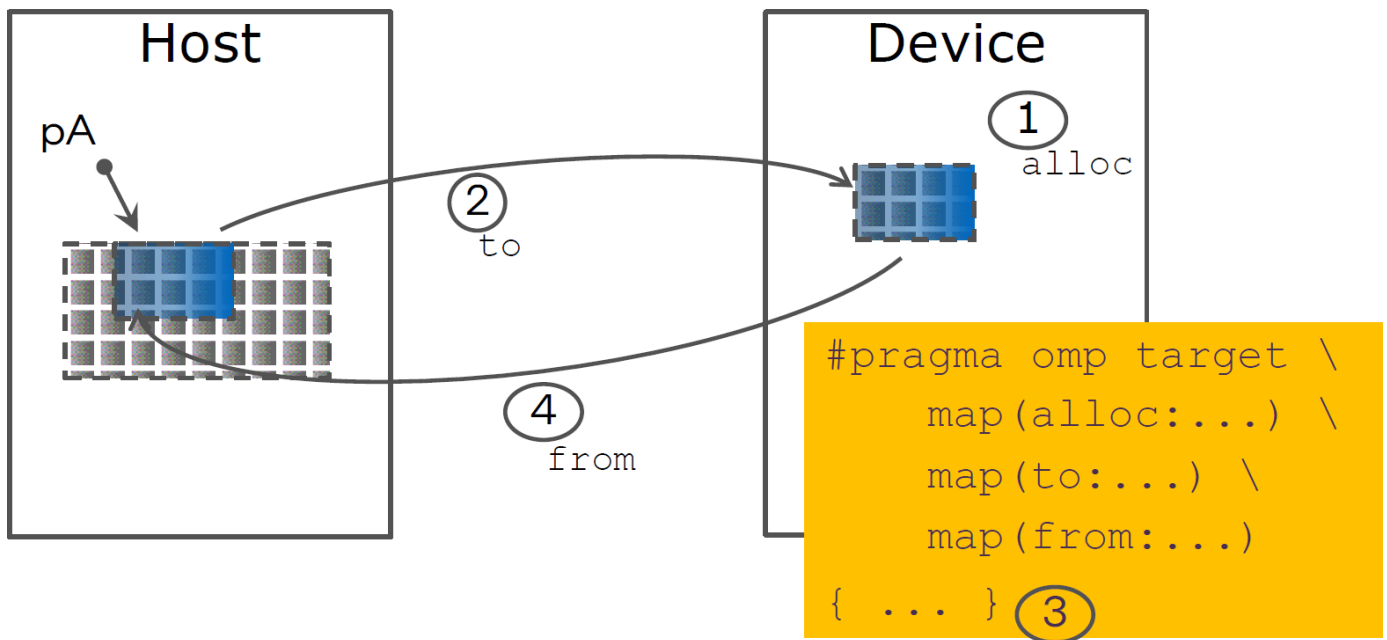


Figure 4: Fig: Transferencia entre GPU-CPU

```
private(b, i, j, k, l)
for (b = 0; b < BLOCKS; b++) {
    for (i = 0; i < P; i++) {
        for (j = 0; j < P; j++) {
            for (k = 0; k < P; k++) {
                .....
            }
        }
    }
}

/* offload kernel #2 */
#pragma omp target teams distribute parallel for collapse(4) \
    map(to: u[0:SIZE], dx[0:P * P]) map(tofrom: w[0:SIZE]) \
    private(b, i, j, k, l)
for (b = 0; b < BLOCKS; b++) {
    for (i = 0; i < P; i++) {
        for (j = 0; j < P; j++) {
            for (k = 0; k < P; k++) {
                .....
            }
        }
    }
}
...

```

```
user@system:~$ OMP_TARGET_OFFLOAD=MANDATORY LIBOMPTARGET_DEBUG=1 ./a.out &> test_no_target_enter_exit_data
user@system:~$ grep "omptarget --> Moving" test_no_target_enter_exit_data.debug
omptarget --> Moving 8192 bytes (hst:0x00007ffea8b9a1c0) -> (tgt:0x00007598cec50000)
omptarget --> Moving 256 bytes (hst:0x00007ffea8b9e1c0) -> (tgt:0x00007598cc6d0000)
omptarget --> Moving 8192 bytes (tgt:0x00007598cec52000) -> (hst:0x00007ffea8b9c1c0)
omptarget --> Moving 8192 bytes (hst:0x00007ffea8b9c1c0) -> (tgt:0x00007598cec52000)
omptarget --> Moving 8192 bytes (tgt:0x00007598cec52000) -> (hst:0x00007ffea8b9c1c0)
```


[!WARNING]

¿hace falta en el kernel#2 enviar 'u', 'dx' y 'w'?

- Usar `omp target enter data map` y `omp target exit data map` como en el código “test_target_enter_exit_data.cpp”

```
...
/* map data to device. alloc for w avoids map(tofrom: w[0:SIZE]) on target by default. */
#pragma omp target enter data map(to: u[0:SIZE], dx[0:P * P]) \
    map(alloc: w[0:SIZE])

/* offload kernel #1 */
...

/* offload kernel #2 */
...

#pragma omp target exit data map(from: w[0:SIZE])
...
```

Ejercicio 3: Inferencia CNN con OpenMP (entrega)

El objetivo de esta práctica es modificar un código en C++ que realiza inferencia sobre imágenes del dataset MNIST, para añadir soporte de **paralelismo heterogéneo** mediante **OpenMP-offloading**.

El punto de partida es el fichero fuente `cnn.cpp`, el cual sigue el siguiente flujo de trabajo:

1. **Carga los pesos** del modelo previamente entrenado.
2. **Infiere el modelo** sobre una imagen del dataset MNIST.
3. **Devuelve la predicción**, reconociendo el número escrito a mano (del 0 al 9).

Arquitectura del modelo

El modelo implementado es una red neuronal convolucional (CNN) **sencilla**, compuesta por las siguientes capas:

- Conv2D: una convolución con una máscara de tamaño **3x3**.
- ReLU: función de activación rectificada.
- MaxPool2D: operación de reducción espacial (submuestreo).
- Fully Connected (FC): capa densa que produce la **probabilidad** de que la imagen corresponda a un dígito entre el **0 y el 9**.

El modelo implementado está basado en una arquitectura similar a propuesta en PyTorch en las siguiente líneas de código:

```
import torch
import torch.nn as nn
# Define the model
class SimpleCNN(nn.Module):
    def __init__(self):
        super(SimpleCNN, self).__init__()
        self.conv1 = nn.Conv2d(in_channels=1, out_channels=1, kernel_size=3, padding=1) # 28x28 -> 28x28
        self.relu = nn.ReLU()
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2) # 28x28 -> 14x14
        self.fcl = nn.Linear(1 * 14 * 14, 10) # Fully Connected Layer (10 classes)
```

La siguiente imagen muestra un diagrama de la arquitectura del modelo seguido que puede resumirse en el procesado de las capas: [Input] → [Conv2d] → [ReLU] → [MaxPool2d] → [FullyConected] → [Output]

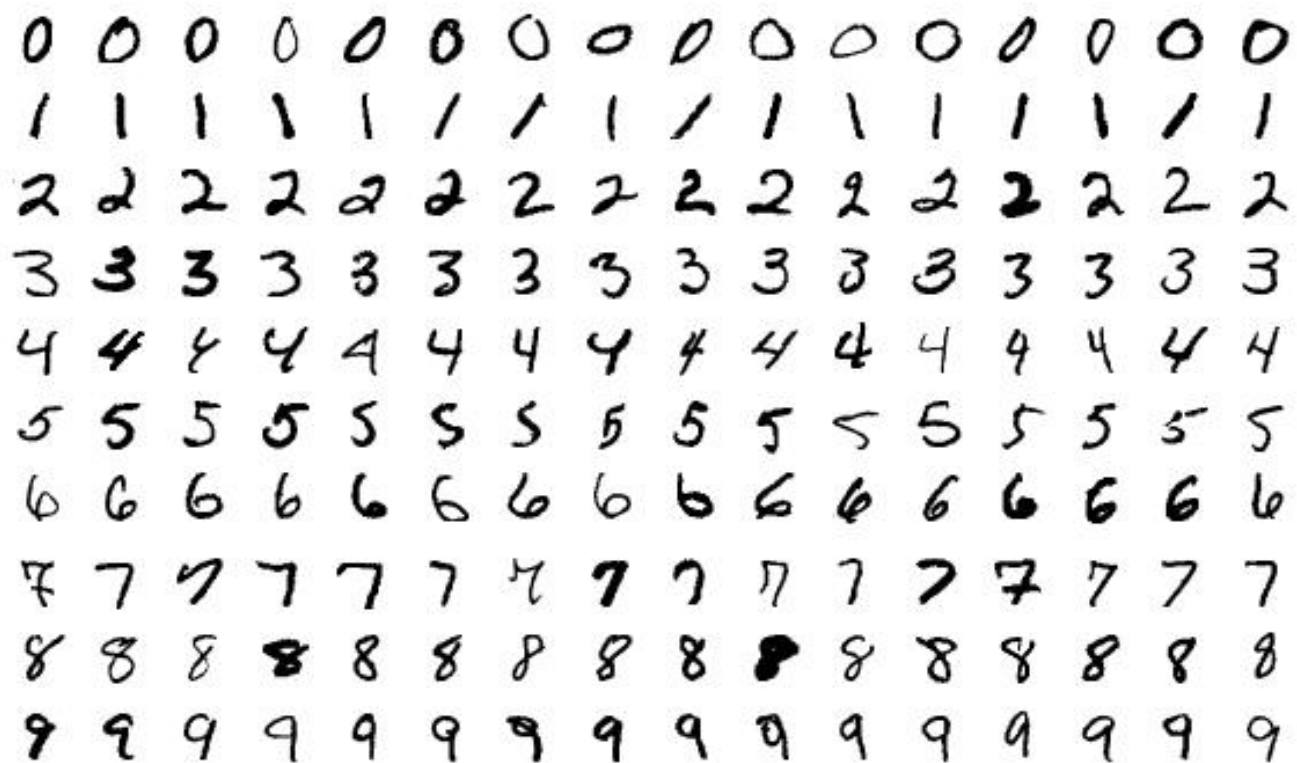


Figure 5: Fig: La base de datos MNIST (Modified National Institute of Standards and Technology) es una gran colección de dígitos escritos a mano. Contiene un conjunto de entrenamiento con 60,000 ejemplos y un conjunto de prueba con 10,000 ejemplos.

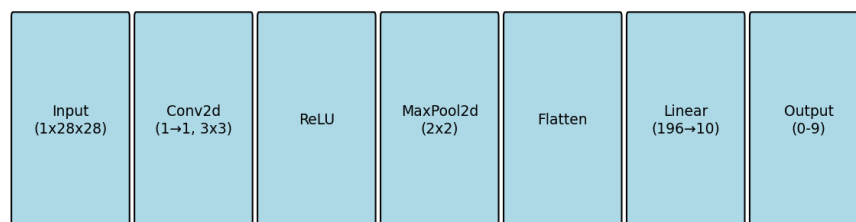


Figure 6: Fig: Arquitectura del modelo CNN

Tareas del alumno

El estudiante debe **optimizar y paralelizar** las funciones críticas del proceso de inferencia utilizando **OpenMP target offloading**, para ello se centrará en el código que implementan las funciones Conv2d+ReLU, MaxPool2d y FullyConected que aparecen resaltadas a continuación:

```
conv2d(input, conv_out, conv1_weights, height, width, *conv1_biases);  
  
maxpool2d(conv_out, pooled, height, width);  
  
fully_connected(pooled, fcl_weights, fcl_biases, output, size_fcl_biases, fc_input_size);
```

Para identificar con más sencillez dicha parte del código, estas líneas corresponde a la monitorización del tiempo de ejecución del modelo entre las llamadas a la función `get_ms()` que captura el tiempo del sistema.

Recomendaciones propuestas

Para facilitar una implementación eficaz y comprensible, se presentan las siguientes recomendaciones:

1. Separación en kernels independientes:

Cada una de las funciones (`conv2d`, `maxpool2d`, `fully_connected`) tiene dependencias de datos con las demás. Por ello, deben ser tratadas como **kernels independientes** en el contexto de OpenMP target offloading. Es decir, cada función debe ser procesada offloading por separado, respetando el orden lógico de ejecución.

2. Minimización de transferencias CPU-GPU:

Recuerda que transferir datos entre CPU y GPU tiene un coste elevado. OpenMP permite controlar esto con la cláusula `map`. Por ejemplo, `conv2d` genera el array `conv_out`, que es inmediatamente usado por `maxpool2d`. **Evita volver a mapear datos que ya están en el dispositivo.**

3. Aprovechamiento del paralelismo en la GPU:

La cláusula `target` por sí sola solo descarga el kernel al dispositivo. Para sacar el máximo rendimiento, es fundamental añadir otras cláusulas como `teams`, `distribute`, `parallel for`, `simd` etc., que permiten explotar toda la jerarquía de paralelismo de la GPU.

4. Uso de `collapse` para fusionar bucles:

En muchos algoritmos, los bucles anidados son candidatos a ser “fusionados” para paralelizar más iteraciones de manera efectiva. Usa la cláusula `collapse(N)` para fusionar N niveles de bucles cuando sea coherente con la lógica del algoritmo.

5. Precaución con las reducciones:

Las operaciones de reducción (sumas acumulativas, o cálculo de valores máximos; por ejemplo) no suelen ser eficientes en GPU, y a veces tampoco en CPU. **Evítalas si es posible**, aunque eso signifique reducir la forma de expresar el paralelismo, suele tener más impacto en el rendimiento una reducción que una posible ganancia extra por poder expresar más paralelismo.

6. Escalabilidad limitada por el problema:

El modelo CNN utilizado es intencionadamente simple, para que pueda ser optimizado sin mucho tiempo de desarrollo, además la imagen de entrada es bastante pequeña (28x28 píxeles). Por tanto, no esperes grandes mejoras de rendimiento al usar GPU. **El objetivo es más pedagógico-uso de GPU-que de ganancia en el rendimiento.** No obstante si quieres evaluar la escalabilidad del problema está disponible el código fuente `cnn_batch.cpp` que permite inferir 1024 imágenes permitiendo un caso más realista.

Resultado esperado

Se espera que el alumno sea capaz de:

- Implementar correctamente el offloading con OpenMP, que los dígitos reconocidos sean los mismo que la versión secuencial.
- Evaluar los sobrecostos mediante el uso de herramientas de perfilado o herramientas que informen del paralelismo expresado en la GPU, sobre-coste de trasiego de datos. . . .
- Comprender las ventajas del paralelismo heterogéneo en la inferencia de modelos de deep learning.